

```

# =====
# Text Clustering + Sentiment Analysis Pipeline
# =====
# Author: Dev Kinger
# We shall do the following:
# - Extract meaningful vocabulary that we want from company & industry text
# - Cluster semantically similar words
# - Map clusters back to sentences
# - Measure sentiment at cluster × year × entity level
# =====

# -----
# Imports
# -----
import os
import pandas as pd
import numpy as np
import spacy
import torch

from collections import Counter, defaultdict
from sklearn.cluster import KMeans
from sentence_transformers import SentenceTransformer
from nltk.tokenize import sent_tokenize
from transformers import AutoTokenizer, AutoModelForSequenceClassification

# -----
# Configuration
# -----
BASE_FOLDER = r"C:\Users\New\Desktop\S&P Assignment\Outputs"

```

```
COMPANY_FILE = os.path.join(BASE_FOLDER, "company.xlsx")
```

```
INDUSTRY_FILE = os.path.join(BASE_FOLDER, "industry.xlsx")
```

```
MIN_WORD_FREQ = 5
```

```
RANDOM_STATE = 42
```

```
# -----
```

```
# Load NLP Models
```

```
# -----
```

```
print("Loading spaCy model...")
```

```
nlp = spacy.load("en_core_web_sm")
```

```
print("Loading FinBERT model...")
```

```
FINBERT_MODEL = "yiyanghkust/finbert-tone"
```

```
tokenizer = AutoTokenizer.from_pretrained(FINBERT_MODEL)
```

```
finbert = AutoModelForSequenceClassification.from_pretrained(FINBERT_MODEL)
```

```
finbert.eval()
```

```
# =====
```

```
# 1. Vocabulary Extraction
```

```
# =====
```

```
def extract_words_from_file(
```

```
    path,
```

```
    text_col="Section_text",
```

```
    min_freq=MIN_WORD_FREQ
```

```
):
```

```
    """
```

```
    Extracts candidate topic words from a document.
```

```
    Keeps only:
```

- nouns and adjectives
- lowercase lemmas

- non-stopwords
- reasonably frequent words

"""

```
df = pd.read_excel(path)
texts = df[text_col].dropna().astype(str)
```

```
doc = nlp(" ".join(texts))
```

```
words = []
for tok in doc:
    if (
        tok.is_alpha
        and not tok.is_stop
        and tok.pos_ in {"NOUN", "ADJ"}
        and tok.ent_type_ == ""
        and tok.lemma_.islower()
        and len(tok.lemma_) > 2
    ):
        words.append(tok.lemma_)
```

```
freq = Counter(words)
vocab = [w for w, c in freq.items() if c >= min_freq]
```

```
return vocab
```

```
print("Extracting vocabularies...")
company_vocab = extract_words_from_file(COMPANY_FILE)
industry_vocab = extract_words_from_file(INDUSTRY_FILE)

vocab = sorted(set(company_vocab + industry_vocab))
```

```

print(f"Total vocabulary size: {len(vocab)}")

# =====

# 2. Word Embeddings + Initial Clustering

# =====

print("Encoding words using SentenceTransformer...")
embedder = SentenceTransformer("all-MiniLM-L6-v2")
embeddings = embedder.encode(vocab, show_progress_bar=True)

print("Embedding matrix shape:", embeddings.shape)

K_INITIAL = 12
kmeans = KMeans(
    n_clusters=K_INITIAL,
    random_state=RANDOM_STATE,
    n_init=20
)

cluster_ids = kmeans.fit_predict(embeddings)

cluster_df = pd.DataFrame({
    "word": vocab,
    "cluster_id": cluster_ids
}).sort_values("cluster_id")

cluster_df.to_excel("word_clusters.xlsx", index=False)
print("Saved word_clusters.xlsx")

# Quick manual inspection
clusters = defaultdict(list)
for w, cid in zip(vocab, cluster_ids):

```

```

clusters[cid].append(w)

for cid in sorted(clusters):
    print(f"\nCluster {cid} sample:", clusters[cid][:10])

# =====
# 3. Prune Generic / Boilerplate Words
# =====

JUNK_WORDS = {
    "good", "bad", "positive", "negative", "strong", "weak",
    "healthy", "stable", "aggressive", "overall", "support",
    "high", "low", "level", "large", "small",
    "year", "new", "limited", "entity", "factor", "analysis"
}

pruned_vocab = []
pruned_embeddings = []

for w, emb in zip(vocab, embeddings):
    if w not in JUNK_WORDS:
        pruned_vocab.append(w)
        pruned_embeddings.append(emb)

pruned_embeddings = np.vstack(pruned_embeddings)

print("Original vocab size:", len(vocab))
print("Pruned vocab size:", len(pruned_vocab))

# =====
# 4. Refined Clustering
# =====

```

```

K_REFINED = 10
kmeans_refined = KMeans(
    n_clusters=K_REFINED,
    random_state=RANDOM_STATE,
    n_init=20
)

refined_ids = kmeans_refined.fit_predict(pruned_embeddings)

refined_cluster_df = pd.DataFrame({
    "word": pruned_vocab,
    "cluster_id": refined_ids
}).sort_values("cluster_id")

refined_cluster_df.to_excel("refined_word_clusters.xlsx", index=False)
print("Saved refined_word_clusters.xlsx")

```

```

# =====
# 5. Build Cluster → Sentence Corpus
# =====

```

```

def get_sentences(text):
    doc = nlp(str(text))
    return [
        s.text.strip()
        for s in doc.sents
        if len(s.text.strip()) > 25
    ]

```

```

def build_cluster_sentence_corpus(df, entity_label, cluster_words):

```

```

rows = []

for year in sorted(df["Year"].unique()):
    df_year = df[df["Year"] == year]

    cluster_sentences = {cid: [] for cid in cluster_words}

    for _, row in df_year.iterrows():
        sentences = get_sentences(row["Section_text"])

        for sent in sentences:
            tokens = {
                tok.lemma_.lower()
                for tok in nlp(sent)
                if tok.is_alpha and not tok.is_stop
            }

            for cid, words in cluster_words.items():
                if tokens & words:
                    cluster_sentences[cid].append(sent)

    for cid, sents in cluster_sentences.items():
        if sents:
            rows.append({
                "entity": entity_label,
                "year": year,
                "cluster_id": cid,
                "sentences": " ".join(sents)
            })

return pd.DataFrame(rows)

```

```

print("Loading input data...")
company_df = pd.read_excel(COMPANY_FILE)
industry_df = pd.read_excel(INDUSTRY_FILE)

CLUSTER_WORDS = (
    refined_cluster_df
    .groupby("cluster_id")["word"]
    .apply(set)
    .to_dict()
)

company_corpus = build_cluster_sentence_corpus(
    company_df, "company", CLUSTER_WORDS
)

industry_corpus = build_cluster_sentence_corpus(
    industry_df, "industry", CLUSTER_WORDS
)

cluster_corpus = pd.concat(
    [company_corpus, industry_corpus],
    ignore_index=True
)

cluster_corpus.to_excel("cluster_sentence_corpus.xlsx", index=False)
print("Saved cluster_sentence_corpus.xlsx")

# =====
# 6. Sentence-Level Expansion
# =====

rows = []

```



```

for _, row in cluster_corpus.iterrows():
    for sent in sent_tokenize(str(row["sentences"])):
        if len(sent.strip()) > 25:
            rows.append({
                "entity": row["entity"],
                "year": row["year"],
                "cluster_id": row["cluster_id"],
                "sentence": sent.strip()
            })

```

```

sentence_df = pd.DataFrame(rows)
print("Total sentence-level rows:", len(sentence_df))

```

```

# =====
# 7. FinBERT Sentiment Scoring
# =====

```

```

def finbert_score(sentence):
    inputs = tokenizer(
        sentence,
        return_tensors="pt",
        truncation=True,
        max_length=512
    )

    with torch.no_grad():
        outputs = finbert(**inputs)
        probs = torch.softmax(outputs.logits, dim=1)[0].numpy()

    return probs[0], probs[1], probs[2]

```

```
sentiment_rows = []
for _, row in sentence_df.iterrows():
    pos, neg, neu = finbert_score(row["sentence"])
```

```
    sentiment_rows.append({
        "entity": row["entity"],
        "year": row["year"],
        "cluster_id": row["cluster_id"],
        "positive": pos,
        "negative": neg,
        "neutral": neu,
        "net_sentiment": pos - neg
    })
```

```
sentiment_df = pd.DataFrame(sentiment_rows)
```

```
# =====
# 8. Aggregation
# =====
```

```
final_agg = (
    sentiment_df
    .groupby(["entity", "year", "cluster_id"])
    .agg(
        n_sentences=("net_sentiment", "count"),
        avg_positive=("positive", "mean"),
        avg_negative=("negative", "mean"),
        avg_neutral=("neutral", "mean"),
        net_sentiment=("net_sentiment", "mean")
    )
    .reset_index()
)
```

```
final_agg.to_excel("final_cluster_sentiment.xlsx", index=False)  
print("Saved final_cluster_sentiment.xlsx")
```

```
print("Pipeline completed successfully.")
```